

vAI Navigator — iOS App Build Brief

OVERVIEW

Build a native iOS app called **vAI Navigator** — a beautifully designed hiking and walking progress tracker that converts real workouts into visual progress along virtual destination routes. This app is part of the Vayla AI product family; it should feel modern, premium, and data-driven.

The core idea: a user sets a virtual route between any two points on Earth (e.g., Sandpoint, ID → Denver, CO). The app pulls HealthKit workout data, converts each hike/walk into elevation-adjusted equivalent miles, and advances a traveler marker along a real road route. Every workout drops a pin at the precise geographic location along that route. Users can switch between Standard, Satellite, Hybrid, and Terrain map views, and tap any pin to open an embedded Google Street View panorama of that location.

Multiple concurrent projects supported. Each has independent progress.

TECHNICAL STACK

- **Platform:** iOS 17+ (SwiftUI, Swift 5.9+)
- **Persistence:** SwiftData
- **Maps:** Google Maps SDK for iOS (not MapKit)
- **Places:** Google Places SDK for iOS (address autocomplete)
- **Directions:** Google Directions API (HTTP)
- **Geocoding:** Google Geocoding API (reverse geocoding for "near [city]")
- **Health:** HealthKit framework
- **Concurrency:** async/await throughout; no completion handlers unless required by Apple APIs
- **Architecture:** MVVM with @Observable view models, dependency injection via environment
- **Minimum deployment:** iOS 17.0

DEPENDENCIES (Swift Package Manager)

Add these SPM packages:

- <https://github.com/googlemaps/ios-maps-sdk> (GoogleMaps)
- <https://github.com/googlemaps/ios-places-sdk> (GooglePlaces)

Do NOT use CocoaPods.

API KEY HANDLING

- Create `Secrets.swift` with:

```
swift enum Secrets { static let googleMapsAPIKey = "YOUR_KEY_HERE" }
```

- Add `Secrets.swift` to `.gitignore`
 - Commit `Secrets.swift.template` with placeholder and setup instructions
 - In `App.init()` or `AppDelegate`: call
`GMSServices.provideAPIKey(Secrets.googleMapsAPIKey)` and
`GMSPPlacesClient.provideAPIKey(Secrets.googleMapsAPIKey)`
 - README must include a "Setup" section explaining how to: 1. Create a Google Cloud project 2. Enable required APIs (Maps SDK for iOS, Places SDK for iOS, Directions, Geocoding, Street View Static / Panorama) 3. Create an API key restricted to the app's bundle ID 4. Copy `Secrets.swift.template` to `Secrets.swift` and paste the key
-

DATA MODEL (SwiftData)

Project

```
@Model final class Project {
    @Attribute(.unique) var id: UUID
    var name: String
    var startLabel: String          // human-readable, e.g., "Sandpoint, ID"
    var endLabel: String
    var startLatitude: Double
    var startLongitude: Double
    var endLatitude: Double
    var endLongitude: Double
    var encodedPolyline: String    // Google's encoded polyline format
    var totalRouteMiles: Double
    var createdAt: Date
    var isActive: Bool            // exactly one project active at a time
    var isArchived: Bool
    var completedAt: Date?
    @Relationship(deleteRule: .cascade) var workouts: [Workout] = []
}
```

Workout

```
@Model final class Workout {
    @Attribute(.unique) var id: UUID
    var healthKitUUID: UUID        // HKWorkout.uuid, dedup key
    var date: Date
    var rawDistanceMiles: Double
    var elevationGainFeet: Double  // 0 if unavailable
    var adjustedMiles: Double      // computed: raw + (gain / 500)
    var routePositionMiles: Double // cumulative adjusted miles at this point
    var coordinateLatitude: Double
    var coordinateLongitude: Double
    var activityTypeRaw: Int       // HKWorkoutActivityType.rawValue
    var project: Project?
}
```

Qualifying HealthKit workout types

Only these count:

- `.hiking`
- `.walking`
- `.running`

Filter at the HealthKit query level.

CORE ALGORITHMS

Distance adjustment (elevation-weighted)

```
adjustedMiles = rawMiles + (elevationGainFeet / 500.0)
```

If `elevationGainFeet` is nil or zero, `adjustedMiles = rawMiles`. Display BOTH raw and adjusted in the UI. Never hide the raw value.

Route generation

1. Geocode start and end labels → coordinates (via Google Geocoding API)
2. Call Google Directions API: `https://maps.googleapis.com/maps/api/directions/json?origin=LAT,LNG&destination=LAT,LNG&mode=driving&key=KEY`
3. Parse `routes[0].overview_polyline.points` (encoded string)
4. Parse `routes[0].legs[].distance.value` (meters) → convert to miles for total
5. Store encoded polyline string directly in Project
6. Decode on-demand when rendering or computing pin positions

Polyline decoding

Implement Google's standard polyline decoding algorithm: <https://developers.google.com/maps/documentation/utilities/polylinealgorithm> Produces `[CLLocationCoordinate2D]`.

Pin coordinate computation

Given a target cumulative distance D (miles) along a route:

1. Decode polyline to coordinate array
2. Iterate segments; for each pair of consecutive coordinates, compute haversine distance

3. Accumulate until the next segment would exceed D
4. Linearly interpolate between the two endpoints to find the exact point
5. Return `CLLocationCoordinate2D`

Implement haversine properly — Earth radius 3958.8 miles, lat/lng in radians.

Incremental position updates

When a new workout is added to a project:

1. `routePositionMiles = sum of all prior workouts.adjustedMiles` for that project (sorted by date ascending)
2. Compute coordinate via algorithm above
3. Persist to the Workout record

If a workout is deleted or reassigned, recompute all subsequent workouts' positions for that project.

HEALTHKIT INTEGRATION

Permission

On first launch (or when entering any view that needs workouts), request:

- `HKObjectType.workoutType()`
- `HKQuantityType.quantityType(forIdentifier: .distanceWalkingRunning)`
- `HKQuantityType.quantityType(forIdentifier: .elevationAscended)` (if available, iOS 11.2+; otherwise skip gracefully)

Handle denied permission with a clear explainer screen and a link to Settings.

Fetching workouts

Query `HKSampleQuery` with:

- Predicate: `HKQuery.predicateForWorkouts(with: .hiking)` OR `.walking` OR `.running` (combine with `NSCompoundPredicate`)
- Sort: date ascending
- Limit: none (paginate if needed)
- Since last sync timestamp (stored in `@AppStorage`)

For each returned `HKWorkout`:

- Distance: `workout.totalDistance?.doubleValue(for: HKUnit.mile())`

- Elevation: query `HKQuantityType.quantityType(forIdentifier: .elevationAscended)` samples with predicate `HKQuery.predicateForSamples(withStart: workout.startDate, end: workout.endDate)` and sum them, convert meters to feet ($\times 3.28084$)
- Dedup against existing `Workout.healthKitUUID`

Sync flow

After sync, present a review sheet listing new workouts. For each, user can:

- Assign to active project (default)
- Reassign to a specific project
- Skip (don't import)
- Mark all and "Assign all to [active project]"

Settings toggle: "Auto-assign new workouts to active project" — if on, skip the review sheet and auto-assign silently.

VIEW HIERARCHY

App Structure

```
vAINavigatorApp
├─ rootView (SwiftUI)
|   └─ ProjectListView (if any projects exist)
|   └─ OnboardingView (first launch only)
```

1. OnboardingView

Three quick screens:

1. **Hero:** "Walk your way somewhere meaningful." Animated map artwork.
2. **Permissions:** HealthKit permission request with explainer copy.
3. **First Project:** "Let's set your first destination." Goes to `NewProjectView`.

Skip available except on screen 2.

2. ProjectListView

- Large title: "vAI Navigator"
- Segmented control: "Active" / "Archived"

- List of projects as cards:
- Project name (large)
- Start → End subtitle with map pin icons
- Progress bar (animated on appear)
- "247 of 1,150 mi · 21%"
- Small thumbnail of route (static Google Maps image API, optional — skip if rate concerns)
- Tap card → ProjectDetailView
- Floating action button (+) → NewProjectView
- Empty state with illustration and "Create your first route" CTA
- Pull-to-refresh triggers HealthKit sync

3. ProjectDetailView

Full-screen Google map with overlay controls:

Map:

- `GMSMapView` via `UIViewRepresentable`
- Full route polyline: gray, 4pt, untraveled portion
- Traveled portion polyline: gradient orange-to-green, 6pt, on top
- Current position marker: custom icon (hiker silhouette in circle)
- Workout pins: small colored dots (orange), tap → WorkoutPinDetailView
- Camera: fit bounds to full route on first appear, allow free pan/zoom
- "Recenter" button if user pans away

Top overlay (translucent card):

- Project name
- "247 / 1,150 mi · 21% complete"
- "Near Rock Springs, WY" (reverse geocoded from current position)
- "42 workouts logged"
- Subtle close button → back to list

Bottom overlay (draggable sheet):

- Three detents: small (stats), medium (workout list), large (full screen)
- Small detent content:
 - Adjusted miles this week
 - Streak (consecutive days with qualifying workouts)
 - Projected completion date (at 30-day avg pace)
- Medium/large detent content:
 - Workout list, chronological, newest first

- Each row: date, raw mi, elevation, adjusted mi
- Swipe actions: Reassign, Delete

Map type control (floating button, top right):

- Compact menu: Normal / Satellite / Hybrid / Terrain
- Persist last choice per project

4. NewProjectView

- Name field
- Start location: Google Places autocomplete search field
- End location: Google Places autocomplete search field
- As soon as both are selected, preview the route on a small embedded GMSMapView
- Display total route distance
- Estimated completion at user's current pace (if any prior workout data)
- Save button (disabled until route successfully generated)
- Loading state while Directions API is called
- Error handling: no route found, API failure, rate limit → user-friendly message

5. WorkoutPinDetailView

Presented as a sheet when a workout pin is tapped:

- Workout date, formatted
- Activity type icon (hiking boot, walker, runner)
- Stats card:
 - Raw distance: 6.2 mi
 - Elevation gain: 2,400 ft
 - Adjusted: 11.0 equivalent mi
- "You reached this point in your journey"
- Embedded `GMSPanoramaView` at the coordinate
- Check availability first via `GMSPanoramaService.requestPanorama(near:)`
- If no panorama within ~50km, show a fallback Google Static Maps satellite image at that coordinate
- Action menu: Reassign project, Delete workout

6. SettingsView

- Auto-assign toggle
- Default map view (Normal / Satellite / Hybrid / Terrain)

- Elevation adjustment divisor (default 500, editable for power users: 333 for Naismith-strict, 500 for standard, 1000 for minimal bonus)
- HealthKit sync status and "Sync now" button
- About section with version, privacy policy link
- "Reset all data" with confirmation (nukes SwiftData store)

VISUAL DESIGN

Identity

vAI Navigator is part of the Vayla AI family. Visual personality: confident, precise, outdoorsy-modern — think Arc'teryx meets Linear.

Color palette

Primary:	#2D6A4F	(deep forest green – brand anchor)
Accent warm:	#E07A3C	(warm terracotta – progress, pins)
Accent cool:	#4A7B9D	(slate blue – secondary actions)
Background:	#FAFAF7	(warm off-white, light mode)
Background dk:	#0E1412	(deep dark green-black, dark mode)
Surface:	#FFFFFF / #1A201E	
Text primary:	#1A201E / #F5F2EC	
Text secondary:	#6B7570 / #A8B0AA	
Divider:	#E5E3DC / #2A312D	

Traveled route uses a gradient from `#E07A3C` (start) to `#2D6A4F` (end of traveled portion).

Untraveled route: `#A8B0AA` at 50% opacity.

Typography

- System font (SF Pro) throughout for native feel
- Large titles: `.largeTitle.weight(.semibold)`
- Stat numbers: `.system(.title, design: .rounded, weight: .semibold)` — rounded variant for numeric emphasis
- Body: `.body`
- Captions: `.caption.weight(.medium)` in secondary color

Components

- Cards: rounded 16pt corners, subtle shadow (`.shadow(color: .black.opacity(0.08), radius: 12, y: 4)`)
- Buttons: filled primary uses brand green, secondary uses outlined style
- Progress bars: custom rounded bar, animated fill on appear (spring animation, 0.6s)
- Map pins: custom SF Symbol-based markers rendered as UIImage for GMSMarker.icon

Google Maps styling

Apply a custom style JSON to GMSMapView for brand consistency:

- Roads: muted
- Landscape: warm beige in light, deep green-black in dark
- Labels: high contrast
- POIs: hidden (reduce clutter) except for major landmarks
- Include BOTH a light and dark style JSON; switch based on `colorScheme`

Place style JSONs in `Resources/MapStyles/light.json` and `dark.json`.

Animation

- Spring animations on all value changes (`.interpolatingSpring`)
- Map camera animations use `GMSCameraPosition` with `animate(with:)`
- Progress bar fills with ease-out over 0.6s on view appear
- Workout list rows fade in with slight stagger (0.05s per row)

Haptics

- Light impact on project selection
- Success notification on new workout assigned
- Success notification on milestone crossings (25%, 50%, 75%, 100%)

SPECIAL FEATURES

Milestones

When a project crosses 25%, 50%, 75%, or 100% of total route miles:

- Full-screen celebration modal
- "You've reached Rock Springs, WY — halfway to Denver!"

- Reverse-geocoded to find nearest notable city
- Confetti animation (SwiftUI)
- Haptic success
- Option to share as an image (render a card with stats)

Streak tracking

Count consecutive days with at least one qualifying workout. Display in project stats. Reset visually if broken (no doom, just reset).

Pace projection

On the last 30 days of workouts, compute average adjusted miles per day. Project: "At current pace, you'll reach Denver on [DATE]."

Reverse geocoding cache

Cache reverse-geocoding results (coordinate → city) in SwiftData or UserDefaults keyed by rounded coordinate (to 0.01 precision ≈ 1 km) to minimize API calls. TTL 30 days.

ERROR HANDLING

- All API calls wrapped in `do/try/catch` with user-facing error alerts
- Network failures: "Can't reach Google Maps. Check your connection."
- HealthKit denied: explainer screen with "Open Settings" button
- No route found: "We couldn't find a route between those locations."
- Street View unavailable: graceful fallback to satellite static map
- API key missing: developer-facing assertion with setup instructions

INFO.plist

Required keys:

```
<key>NSHealthShareUsageDescription</key>
<string>VAI Navigator reads your workout data to track progress toward your virtual destinations.</string>

<key>NSLocationWhenInUseUsageDescription</key>
<string>Used to show your current location on the map relative to your route.</string>
```

FILE / FOLDER STRUCTURE

```
vAINavigator/  
├─ App/  
│   ├── vAINavigatorApp.swift  
│   └─ rootView.swift  
├─ Models/  
│   ├── Project.swift  
│   └─ Workout.swift  
├─ Services/  
│   ├── HealthKitService.swift  
│   ├── GoogleDirectionsService.swift  
│   ├── GoogleGeocodingService.swift  
│   ├── PolylineDecoder.swift  
│   └─ DistanceCalculator.swift    (haversine, pin interpolation)  
├─ ViewModels/  
│   ├── ProjectListViewModel.swift  
│   ├── ProjectDetailViewModel.swift  
│   └─ NewProjectViewModel.swift  
├─ Views/  
│   ├── Onboarding/  
│   ├── ProjectList/  
│   ├── ProjectDetail/  
│   │   ├── ProjectDetailView.swift  
│   │   ├── GoogleMapViewRepresentable.swift  
│   │   └─ WorkoutPinDetailView.swift  
│   ├── NewProject/  
│   └─ Settings/  
├─ DesignSystem/  
│   ├── Colors.swift  
│   ├── Typography.swift  
│   ├── Components/  
│   │   ├── StatCard.swift  
│   │   ├── ProgressBar.swift  
│   │   └─ ProjectCard.swift  
│   └─ MapStyles/  
│       ├── light.json  
│       └─ dark.json  
├─ Resources/  
│   └─ Assets.xcassets  
├─ Secrets.swift                (gitignored)  
├─ Secrets.swift.template  
└─ README.md
```

SESSION 1 DELIVERABLE

For this first Claude Code session, build:

1. Xcode project scaffold with SwiftUI app lifecycle, iOS 17+ target
2. SPM dependencies added (GoogleMaps, GooglePlaces)
3. `Secrets.swift.template` committed, `Secrets.swift` gitignored
4. `GMSServices.provideAPIKey()` and `GMSPlacesClient.provideAPIKey()` called on launch
5. SwiftData models for Project and Workout
6. HealthKitService with permission request and workout fetching
7. Design system foundation: `Colors.swift`, `Typography.swift`
8. ProjectListView with empty state, card layout, and "+" button (NewProjectView wired but stubbed)
9. Working README with complete setup instructions
10. `.gitignore` properly configured

Defer to subsequent sessions:

- Map rendering and pin placement
- NewProjectView with Places autocomplete
- Street View integration
- Milestones and animations

TONE AND CODE QUALITY

- Write clean, idiomatic Swift 5.9+
- Use `@Observable` (not `ObservableObject`) for view models where iOS 17+ allows
- Prefer `async/await` over callbacks
- Document public APIs with `///` comments
- No force unwraps except in truly impossible cases (and document why)
- Use `guard` for early returns
- Keep views under 150 lines — extract subviews liberally
- Keep services stateless where possible, inject via environment